# Cut Your Losses in Large-Vocabulary Language Models

Cut Cross-Entropy (CCE)

Разбор статьи

Сухова Софья

Erik Wijmans, Brody Huval, Alexander Hertzberg, Vladlen Koltun, Philipp Krähenbühl. Apple, ICLR 2025.

https://arxiv.org/abs/2411.09009

## Основная идея

Для каждой позиции во время обучения правильный следующий токен известен из данных.

**Цель обучения – увеличить вероятность правильного токена относительно остальных.** Большого логита самого по себе недостаточно: важны и логиты других токенов словаря.

Обычная реализация сохраняет полную матрицу логитов. Но для одного значения лосса нужны только целевой логит и **общий** вклад всех токенов в знаменатель софтмакса.

**Идея CCE: учесть весь словарь, но не хранить отдельные логиты. На форварде сохранить нужные числа, на бэкварде пересчитать детали блоками.**

## Что вычисляется при обучении

Блоки трансформера вычисляют скрытые состояния для $N$ предсказываемых позиций во всем батче. Здесь позиции – места в обучающих последовательностях. Выходной слой вычисляет логиты для каждой позиции и каждого из $V$ токенов словаря.

| | |
|---|---|
| $E = [e_1, \ldots, e_N] \in \mathbb{R}^{D \times N}$ | Скрытые состояния; $D$ – их размерность. |
| $C = [c_1, \ldots, c_V] \in \mathbb{R}^{D \times V}$ | Веса выходного слоя: один вектор на токен. |
| $x_i \in \{1, \ldots, V\}$ | Правильный следующий токен для позиции $i$. |

$$Z = C^\top E \in \mathbb{R}^{V \times N}, \qquad z_{vi} = c_v^\top e_i, \qquad p_{vi} = \frac{\exp(z_{vi})}{\sum_u \exp(z_{ui})}$$

$Z$ **– матрица логитов**: столбец соответствует позиции $i$, строка – токену $v$. Логит $z_{vi}$ – значение до софтмакса; $p_{vi}$ – вероятность после него.

## Почему матрица логитов вообще становится проблемой

Для $N = 8192$ позиций и $V = 256\,000$ токенов словаря:

$$NV = 2\,097\,152\,000 \text{ логитов}, \qquad 2NV \text{ байт} \approx 4{,}19 \text{ ГБ} \quad \text{в bfloat16}$$

Это только одна матрица. Обычное вычисление также создает вероятности и промежуточные буферы; некоторые операции используют 32-битные числа.

Длинный контекст увеличивает $N$, большой словарь – $V$. Объем памяти под эти матрицы растет как $O(NV)$, даже если трансформер уже использует экономные механизмы внимания и пересчет активаций.

**Значение кросс-энтропии для одной позиции – одно число. Зачем ради него хранить целый столбец из $V$ логитов?**

## Кросс-энтропии достаточно двух чисел на позицию

Для правильного следующего токена $x_i$:

$$L_i = -\log p_{x_i,i} = -\log \frac{\exp(z_{x_i,i})}{\sum_v \exp(z_{vi})}$$
$$= \underbrace{\log \sum_v \exp(z_{vi})}_{\text{LSE}_i} - \underbrace{c_{x_i}^\top e_i}_{\text{целевой логит}}$$

**Целевой логит:** одно скалярное произведение с весами правильного токена.

**LSE (log-sum-exp):** логарифм суммы экспонент логитов всего словаря, включая правильный токен. Это общий вклад токенов в знаменатель софтмакса.

$L_i$ **– число, не столбец. Для** $N$ **позиций получаем** $N$ **значений лосса. Отдельные вероятности всех** $V$ **токенов для этого хранить не нужно.**

## Что такое блок логитов

Вся матрица $Z$ имеет $V$ строк токенов словаря и $N$ столбцов позиций. Для очередного вычисления выбираем группу $J$ из $V_B$ токенов словаря и группу $I$ из $N_B$ позиций.

$$\underbrace{C_J^\top}_{V_B \times D} \quad \underbrace{E_I}_{D \times N_B} = \underbrace{A_{JI}}_{V_B \times N_B}$$

$A_{JI}$ – маленькая матрица логитов: только выбранные токены и выбранные позиции. Размеры $V_B$ и $N_B$ выбираются под GPU.

Для каждого ее столбца вычисляем одно значение LSE по текущей группе $J$. Из $V_B \times N_B$ логитов остаются $N_B$ чисел.

**Не создаем большую матрицу – сразу вычисляем только нужный фрагмент; затем переходим к следующему.**

## Почему это экономит память

**HBM** – большая основная память GPU. **SRAM** – небольшая быстрая локальная память GPU.

| | Обычная реализация | CCE |
|---|---|---|
| Блок логитов | Вычислить и записать в полную матрицу в HBM. | Вычислить в SRAM и сразу получить вклад в LSE. |
| После блока | Его логиты остаются в большой матрице. | Сохранить только LSE; память логитов переиспользовать. |
| Результат | Полную матрицу читает следующая операция. | Полная матрица нигде не собирается. |

Обычное матричное умножение тоже работает блоками. Экономия CCE – в отсутствии полной промежуточной матрицы, а не в самом разбиении. Маленький блок тоже занимает память, но ее объем не растет как $NV$.

## Как из всех блоков получается один LSE

Рассмотрим одну позицию $i$. Для очередной группы токенов словаря $J$ вычисляем логиты и LSE только этой группы:

$$z_{vi} = c_v^\top e_i \quad (v \in J), \qquad a_i = \log \sum_{v \in J} \exp(z_{vi})$$

Для первой группы $r_i = a_i$. Далее объединяем накопленный $r_i$ с LSE новой группы:

$$r_i \leftarrow \log\big(\exp(r_i) + \exp(a_i)\big)$$

Это сложение сумм экспонент, представленных через логарифм. После объединения логиты группы не нужны.

Пройдя весь словарь, получаем полный LSE и лосс:

$$r_i = \log \sum_{v=1}^{V} \exp(z_{vi}) = \mathrm{LSE}_i, \qquad L_i = r_i - t_i, \quad t_i = z_{x_i,i}$$

**На форварде учитывается каждый токен словаря. Для нескольких позиций такой результат накапливается отдельно в каждом столбце.**

# Алгоритм 1: вычислить только целевой логит

**Algorithm 1** Memory-efficient indexed matrix multiplication

**Inputs:** $\mathbb{E} \in \mathbb{R}^{D \times N}$, $\mathbf{C} \in \mathbb{R}^{D \times |V|}$, $\mathbf{x} \in \mathbb{R}^{N}$.
Block sizes $N_B$ and $D_B$.

**Outputs:** $\mathbf{o} = (\mathbf{C}^\top \mathbb{E})_{\mathbf{x}} \in \mathbb{R}^{N}$

**for** blocks $\mathbb{E}_n$, $\mathbf{x}_n$ **do** ▷ Divide $\mathbb{E}$ and $\mathbf{x}$ into blocks of size $D \times N_B$ and $N_B$, respectively
  $\mathbf{o}_n = \mathbf{0}_{N_B}$ ▷ Zero vector of size $N_B$ in on-chip SRAM
  **for** blocks $\mathbb{E}_{n,d}$ **do** ▷ Divide $\mathbb{E}_n$ into blocks of size $D_B \times N_B$
    $\mathbf{c} = \mathbf{C}_{\mathbf{x}_{n,d}}$ ▷ Indexed load into on-chip SRAM
    $\mathbf{o}_n \mathrel{+}= \mathbb{E}_{n,d} \cdot \mathbf{c}$ ▷ Column-wide dot product
  **end for**
  write $\mathbf{o}_n$ ▷ From on-chip SRAM to main GPU memory
**end for**

**Индексы.** $n$ – группа $N_B$ позиций; $d$ – группа $D_B$ координат скрытого состояния.

$c = C_{x_n,d}$ – по индексам $x_n$ загружаются только веса правильных токенов.

$o_n \mathrel{+}= E_{n,d} \cdot c$ – скалярные произведения накапливаются по координатам $D$.

**Запись** $o_n$ – сохранить целевые логиты: по одному на позицию.

**Результат – вектор из $N$ чисел.**

# Алгоритм 2: из блока логитов сразу получить LSE

**Algorithm 2** Memory-efficient linear-log-sum-exp, forward pass

**Inputs:** $\mathbb{E} \in \mathbb{R}^{D \times N}$ and $\mathbb{C} \in \mathbb{R}^{D \times |V|}$.
Block sizes $N_B$, $V_B$, and $D_B$.
**Outputs:** $\mathrm{LSE} = \log \sum_j \exp(C_j^\top \mathbb{E}) \in \mathbb{R}^N$

$\mathrm{LSE} = -\infty_N$ ▷ $-\infty$ vector of size $N$ in main GPU memory
**for** all pairs of blocks $\mathbb{E}_n$, $\mathbf{C}_v$ **do** ▷ Divide $\mathbb{E}$ and $\mathbf{C}$ into blocks of size $D \times N_B$ and $D \times V_B$
  $\mathbf{A}_{nv} = \mathbf{0}_{V_B \times N_B}$ ▷ Zero matrix of size $V_B \times N_B$ in on-chip SRAM
  **for** blocks $\mathbb{E}_{n,d}$, $\mathbf{C}_{v,d}$ **do** ▷ Divide $\mathbb{E}_n$ and $\mathbf{C}_v$ into blocks of $D_B \times N_B$ and $D_B \times V_B$
    $\mathbf{A}_{nv} \mathrel{+}= \mathbf{C}_{v,d}^\top \cdot \mathbb{E}_{n,d}$ ▷ Blockwise matrix multiplication
  **end for**
  $\mathrm{LSE}_{nv} = \log \sum \exp(\mathbf{A}_{nv}^\top)$ ▷ Numerically stable implementation with max
  $\mathrm{LSE}_n = \log(\exp(\mathrm{LSE}_n) + \exp(\mathrm{LSE}_{nv}))$ ▷ Locking thread-safe log-add-exp
**end for**

**Сначала** $A_{nv}$**.** $n$ и $v$ – группы позиций и токенов словаря. Получить их блок логитов размером $V_B \times N_B$ в SRAM.

**Затем** $\mathrm{LSE}_{nv}$**.** Из каждого столбца получить одно число – LSE по текущей группе токенов. Это $N_B$ чисел, а не матрица вероятностей.

**Наконец** $\mathrm{LSE}_n$**.** Объединить с накопленным LSE в HBM. Повторить для остальных групп словаря.

**Одно ядро вычисляет логиты и сразу использует их для LSE. В HBM записываются только значения LSE; память блока логитов используется снова.**

## Лосс

Пусть $\theta$ – веса трансформера, $C$ – веса выходного слоя. Из скрытых состояний и логитов получаем один скалярный лосс:

$$E = f_\theta(\text{контекст}), \qquad Z = C^\top E, \qquad L = \sum_i L_i$$

Зачем градиенты по разным величинам:

**По логитам** $Z$**:** промежуточная производная, из которой получаем градиенты по обоим входам произведения $C^\top E$.

**По весам** $C$**:** нужен для обновления выходного слоя.

**По скрытым состояниям** $E$**:** нужен, чтобы продолжить бэквард через трансформер и получить градиенты по его весам $\theta$.

**Обновляются веса** $C$ **и** $\theta$**.**

## Бэквард: восстановить вероятности и накопить градиенты

Снова вычисляем очередной блок логитов. Для каждой его позиции используем полный LSE, сохраненный на форварде:

$$p_{vi} = \exp(z_{vi} - \mathrm{LSE}_i)$$

Та же вероятность софтмакса.

Производная лосса по логиту:

$$g_{vi} = \frac{\partial L_i}{\partial z_{vi}} = \underbrace{p_{vi}}_{\text{из LSE}} - \underbrace{\mathbf{1}[v = x_i]}_{\text{из целевого логита}}$$

Индикатор равен 1 для правильного токена и 0 для остальных. Если $G = (g_{vi})$, то для $L = \sum_i L_i$:

$$\nabla_E L = CG, \qquad \nabla_C L = EG^\top$$

**CCE накапливает эти произведения по блокам. Полные матрицы вероятностей и $G$ не создаются; вклад блока добавляется к уже накопленным градиентам.**

# Алгоритм 4: пересчитать и добавить вклад в градиенты

**Algorithm 4** Memory-efficient linear-cross-entropy loss, backward pass

**Inputs:** $\mathbb{E} \in \mathbb{R}^{D \times N}$, $\mathbf{C} \in \mathbb{R}^{D \times |V|}$, $\mathrm{LSE} \in \mathbb{R}^{N}$, $\nabla\mathrm{CEL} \in \mathbb{R}^{N}$, and $\mathbf{x} \in \mathbb{R}^{N}$.
Block sizes $N_B$, $V_B$, and $D_B$.
Accuracy threshold $\varepsilon$.
$\mathbf{v} = [1, \ldots, |V|]$.

**Outputs:** $\nabla\mathbb{E} \in \mathbb{R}^{D \times N}$, $\nabla\mathbf{C} \in \mathbb{R}^{D \times |V|}$

**for** all pairs of blocks $\mathbb{E}_n$, $\mathbf{C}_v$ **do** ▷ Divide $\mathbb{E}$ and $\mathbf{C}$ into blocks of size $D \times N_B$ and $D \times V_B$
  $\mathbf{A}_{nv} = \mathbf{0}_{V_B \times N_B}$ ▷ Zero matrix of size $V_B \times N_B$ in on-chip SRAM
  **for** blocks $\mathbb{E}_{n,d}$, $\mathbf{C}_{v,d}$ **do** ▷ Divide $\mathbb{E}_n$ and $\mathbf{C}_v$ into blocks of $D_B \times N_B$ and $D_B \times V_B$
    $\mathbf{A}_{nv}$ += $\mathbf{C}_{v,d}^{\top} \cdot \mathbb{E}_{n,d}$ ▷ Blockwise matrix multiplication
  **end for**
  $\mathbf{S}_{nv} = \exp(\mathbf{A}_{nv} - \mathrm{LSE}_n)$ ▷ Compute the softmax
  $\mathbf{G}_{nv} = \left[\left[\mathbf{v} = \mathbf{x}_n^{\top}\right]\right] - \mathbf{S}_{nv}$ ▷ Gradient of cross-entropy loss wrt. logits
  **if** all($|\mathbf{G}_{nv}| < \varepsilon$) **then**
    **skip** ▷ Skip computation if below desired numerical precision
  **end if**
  **for** blocks $\mathbb{E}_{n,d}$, $\mathbf{C}_{v,d}$ **do** ▷ Divide $\mathbb{E}_n$ and $\mathbf{C}_m$ into blocks of $D_B \times N_B$ and $D_B \times V_B$
    $\nabla\mathbb{E}_{n,d}^{\top}$ += $(\mathbf{G}_{nv} \cdot \nabla\mathrm{CEL}_n)\, \mathbf{C}_{v,d}$ ▷ Locking thread-safe gradient update
    $\nabla\mathbf{C}_{v,d}^{\top}$ += $(\mathbf{G}_{nv} \cdot \nabla\mathrm{CEL}_n)^{\top}\, \mathbb{E}_{n,d}$ ▷ Locking thread-safe gradient update
  **end for**
**end for**

**1. Пересчет $A_{nv}$.** Получить тот же маленький блок логитов умножением $C_v^{\top} E_n$.

**2. Вероятности $S_{nv}$.** Восстановить их из логитов и сохраненного полного LSE.

**3. Целевой токен.** Учесть производную целевого логита. `skip` – дополнительная проверка малых градиентов.

**4. Градиенты $E$ и $C$.** Вычислить вклад блока и добавить к накопленному результату. Затем обработать другие блоки.

В псевдокоде выход записан как $\log p$. Множитель $\nabla$CEL учитывает переход к лоссу $-\log p$ и, если нужно, усреднение по позициям.

# Дополнительное ускорение: фильтрация градиентов

Пересчет логитов на бэкварде добавляет работу.

У нецелевых токенов $g_{vi} = p_{vi}$: малая вероятность означает малый градиент по логиту. Блок градиентов можно пропустить, только если малы **все** его элементы:

$$\max_{v \in J,\ i \in I} |g_{vi}| < \varepsilon \quad \Longrightarrow \quad \text{пропустить обновления градиентов этого блока}$$

Авторы используют $\varepsilon = 2^{-12}$. **Уже сделано:** пересчет логитов и вероятностей для проверки. **Пропускаем:** два матричных умножения с $G$ и добавление их результатов в градиенты.

Для правильного токена $g_{x_i,i} = p_{x_i,i} - 1$. При малой вероятности правильного ответа он близок к $-1$ и не отсекается. Критерий – малый градиент.

**Фильтрация делает бэквард приближенным.**

## Дополнительное улучшение: сортировка словаря

Фильтр пропускает блок, только если **все** его элементы малы. Один значимый элемент в каждом блоке может помешать почти всем пропускам.

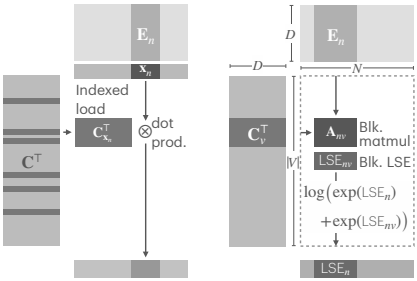
**Идея сортировки:** поставить менее активные токены рядом, чтобы они чаще попадали в целиком пропускаемые блоки. На форварде авторы накапливают средний логит каждого токена:

$$\bar{z}_v = \frac{1}{N} \sum_{i=1}^{N} z_{vi}$$

На бэкварде блоки формируются из токенов с близкими $\bar{z}_v$. Это эвристика, и условие фильтрации все равно проверяется.

Индексы токенов и их веса не меняются, только порядок обработки. Дополнительная память для средних логитов: $O(V)$.

# Итоговая схема: что происходит на форварде




(a) Indexed matmul (forward)

(b) Linear-log-sum-exp, forward pass

**(a) Целевой логит.** `Indexed load`: по индексам $x_n$ загружаем веса правильных токенов. `dot prod.`: скалярное произведение с $E_n$. Внизу – вектор целевых логитов.

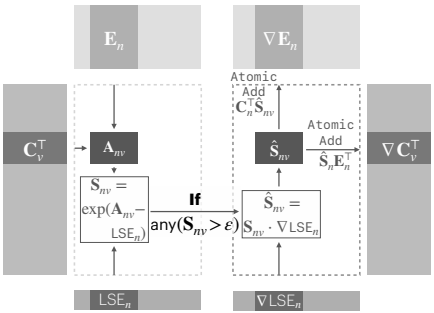
**(b) LSE.** $C_v^\top E_n$ дает маленькую матрицу логитов $A_{nv}$ в SRAM (пунктир). Из каждого столбца получаем LSE по группе токенов. Объединяем с накопленным LSE; результат – нижняя полоска.

Большие массивы – в HBM. Темные участки – выбранные строки $C^\top$ и столбцы $E$.

**Матрица $A_{nv}$ в HBM не записывается. Сохраняем только LSE; локальную память блока переиспользуем.**

Figure 2(a, b).

# Итоговая схема: что происходит на бэкварде




(c) Linear-log-sum-exp, backward pass

$\nabla$ – градиент. Показан бэквард LSE; целевой логит учитывается отдельно.

**Слева: пересчет.** Из $E_n$ и $C_v^\top$ снова получаем $A_{nv}$. Сохраненный полный $\mathrm{LSE}_n$ позволяет восстановить вероятности $S_{nv}$.

**В центре: проверка.** `If` решает, нужно ли вычислять дальнейший вклад этого блока.

**Справа: градиенты.** Добавляем вклад блока к градиентам $E$ и $C$. `Atomic Add` обозначает добавление в общие массивы градиентов при параллельной работе.

**Другие блоки добавят свои вклады в градиент. Пересчет логитов заменяет хранение полной матрицы с форварда.**

Figure 2(c).

## Точность: причины возможной ошибки

**Отсечение малых вкладов.** Порог $2^{-12}$ выбран авторами с учетом округления в bfloat16. Это не гарантия, что сумма всех отброшенных вкладов несущественна. Эффект фильтрации зависит от распределения вероятностей и задачи.

**Накопление в низкой точности.** Много блоков добавляет вклад в одни и те же градиенты. Даже без фильтрации обычное суммирование может терять малые слагаемые.

В опытах предобучения исходный CCE ухудшал качество. Авторы предложили **CCE-Kahan-FullC**: **FullC** сохраняет все вклады в градиент весов $C$; **Кэхэн** уменьшает ошибку округления при накоплении. Кэхэн не возвращает вклады, отброшенные фильтрацией.

**Фильтрация и точность накопления определяют компромисс между скоростью и качеством.**

## Итоги: память и вычисления

| | |
|---|---|
| Убрано | Хранение логитов и вероятностей размера $N \times V$ |
| Сохранено | $O(N)$ значений LSE и лосса; $O(V)$ статистик сортировки. |
| Цена | Повторное вычисление логитов на бэкварде; накопление градиентов из вкладов блоков. |
| Компенсация | Меньше обращений к HBM; фильтрация сокращает обновления градиентов. |

Веса $C$, скрытые состояния $E$ и их градиенты остаются в памяти. Экономим на больших промежуточных матрицах, а не на параметрах модели.

# Результаты: память и время выходного слоя

Gemma 2 2B: $N = 8192$, $V = 256\,000$, $D = 2304$

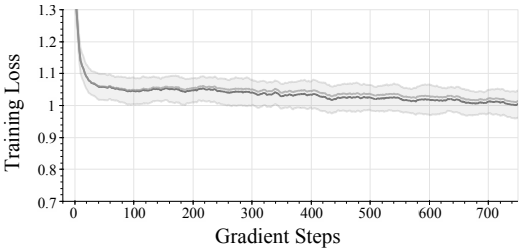
| | Method | Loss | | Gradient | | Loss+Gradient | |
|---|---|---|---|---|---|---|---|
| | | Memory | Time | Memory | Time | Memory | Time |
| | Lower bound | 0.004 MB | | 1,161 MB | | 1,161 MB | |
| 1) | CCE (Ours) | **1 MB** | **46 ms** | **1,163 MB** | 100 ms | **1,164 MB** | 145 ms |
| 2) | Liger Kernels (Hsu et al., 2024)[2] | 1,474 MB | 304 ms | | | 1,474 MB | 304 ms |
| 3) | Torch Tune Team (2024) (8 chunks) | 8,000 MB | 55 ms | 1,630 MB | 115 ms | 9,631 MB | 169 ms |
| 4) | `torch.compile` | 4,000 MB | 49 ms | 12,000 MB | **92 ms** | 16,000 MB | **143 ms** |
| 5) | Baseline | 24,000 MB | 82 ms | 16,000 MB | 122 ms | 28,000 MB | 208 ms |
| 6) | CCE (No Vocab Sorting) | 0.09 MB | 45 ms | 1,162 MB | 115 ms | 1,162 MB | 159 ms |
| 7) | CCE (No Grad. Filter) | 0.09 MB | 45 ms | 1,163 MB | 314 ms | 1,162 MB | 357 ms |
| 8) | CCE-Kahan | 1 MB | 47 ms | 2,325 MB | 114 ms | 2,326 MB | 160 ms |
| 9) | CCE-Kahan-FullC | 1 MB | 47 ms | 2,326 MB | 268 ms | 2,326 MB | 313 ms |
| 10) | CCE-Kahan-FullE | 1 MB | 47 ms | 2,326 MB | 247 ms | 2,326 MB | 292 ms |

**Память:** 24 000 → 1 МБ для вычисления лосса; 28 000 → 1 164 МБ вместе с градиентами. **Время:** CCE — 145 мс; `torch.compile` — 143 мс.

Table 1. A100-SXM4 80 ГБ; PyTorch 2.4.1, CUDA 12.4; пять запусков.

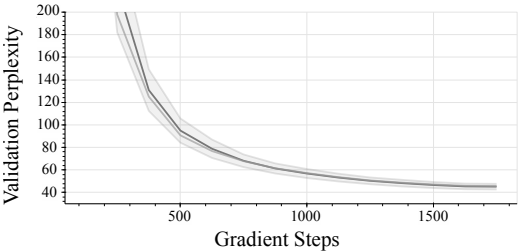
# Результаты: лосс при обучении и сходимость

## Дообучение на Alpaca: CCE




По горизонтали — шаг; по вертикали — лосс при обучении. Кривые CCE и `torch.compile` почти совпали.

## Предобучение: FullC с Кэхэном




5% OpenWebText. По вертикали — проверочная перплексия. Кривые почти совпали.

Темная линия — `torch.compile`, светлая — CCE / CCE-Kahan-FullC. На обоих графиках меньше — лучше.

**Динамика обучения сохранилась в этих опытах; для предобучения нужен более точный режим. Универсальной гарантии сходимости нет.**

Figure 4(a), Figure 5(a). Gemma 2 2B; среднее по пяти запускам и 95%-й интервал.

## Главная идея статьи и приемы, которые ее реализуют

**Идея.** Полная матрица логитов — промежуточный результат. Для кросс-энтропии нужны только целевой логит и LSE.

**Форвард.** Выбор целевых весов совмещается со скалярным произведением. Все токены словаря учитываются в LSE по блокам, без записи полной матрицы логитов в HBM.

**Бэквард.** Из сохраненного LSE и пересчитанных логитов восстанавливаются вероятности. Вклад каждого блока сразу добавляется в градиенты весов и скрытых состояний.

**Дополнительное ускорение.** Пропускаются малые блоки градиентов; сортировка словаря помогает собрать такие блоки.

**Оговорка.** Фильтрация приближает градиенты, а накопление требует контроля округления. Для предобучения нужен более точный режим.